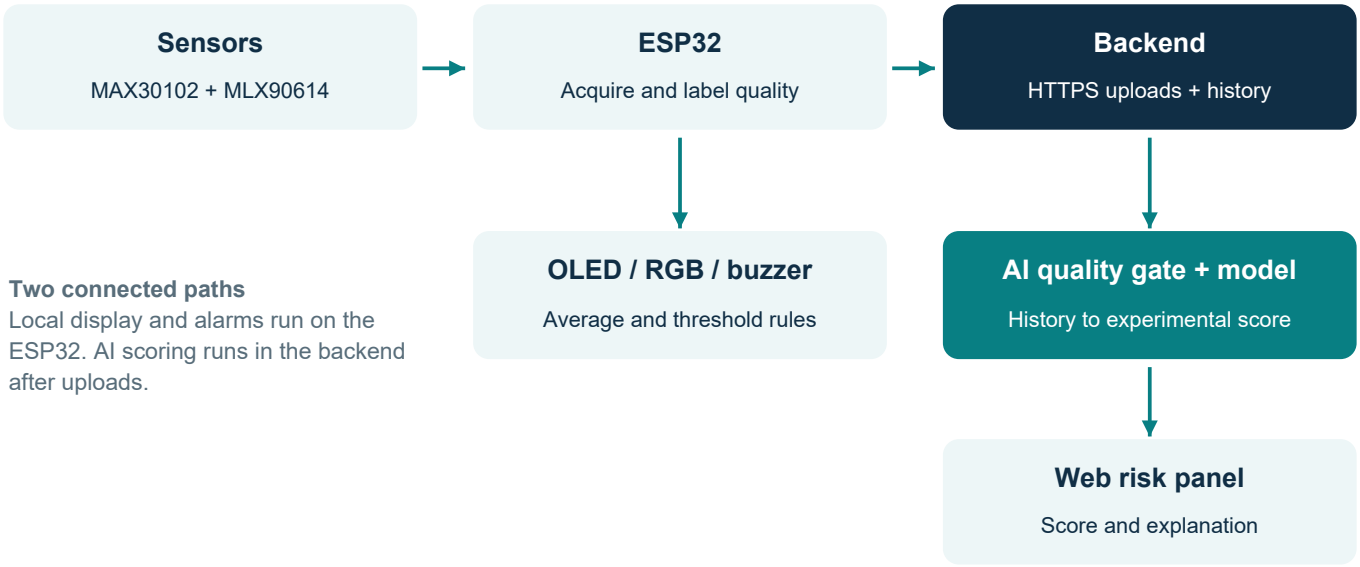
PROJECT REPORT / 24 OCTOBER 2026

How NeoGuard's AI Works

From sensor readings to an experimental early-warning score

NeoGuard combines an ESP32 monitoring device with a hosted backend and web dashboard. Its AI component examines recent heart-rate, oxygen and temperature history to estimate a **simulated low-temperature or low-oxygen event occurring now or within 120 seconds**. Heart rate contributes context to that target. [1,4,5]

CURRENT MODEL SCOPE
The supplied model was trained and tested on simulated sessions. Its score is experimental and is not a clinically validated probability or diagnosis.



Two connected paths
Local display and alarms run on the ESP32. AI scoring runs in the backend after uploads.

Function	Where it runs	What it does
Display smoothing	ESP32	A 10-second heart-rate average, refreshed about every three seconds, makes the local display easier to follow.
Threshold indication	ESP32 and backend rules	Checks configured limits. The local RGB and buzzer operate independently of AI scoring.
AI inference	Backend	Converts qualified recent history into features, applies the saved model and exposes a score to the web app.

What AI adds: it can use both present levels and patterns over time, such as a declining oxygen estimate, rather than relying on one threshold comparison alone. This benefit is supported only by the saved simulation results discussed on page 4. [3,6]

02 / NEOGUARD AI

Which readings may reach the AI?

The device uploads time-stamped sensor values with their session, event identifier and quality label. The backend authenticates the device, validates the batch and stores accepted readings before attempting AI scoring. Buffered historical uploads retain their original session; an old reading is not used as a current warning. [2,7,8]

Two separate quality checks

Stage	Requirements
Device qualification	At least 60 seconds of continuous contact; at least 45 valid observations in the latest minute and 12 in the latest 15 seconds. Recent raw HR spread must be at most 12 bpm, oxygen spread at most 3 percentage points and temperature spread at most 0.4°C.
Backend feature window	Uses only quality=ok observations from up to 120 seconds of history. Each of the three signals needs at least 12 valid observations spanning at least 60 seconds, with gaps no longer than 15 seconds. The latest observation must be valid for every signal.
Additional backend checks	The session must be active, the latest reading must satisfy the configured freshness limit, the model must load, and active temperature/oxygen alert thresholds must match the saved model contract. Duplicate observation times invalidate a feature window.

The device sends unqualified live estimates as **quality=poor**; fully qualified measurements use **quality=ok**. Making the OLED display smoother does not change that label. Initial scoring can therefore require roughly two minutes of clean data: first device qualification, then sufficient qualified history. It can take longer when contact or quality is interrupted. [2,3,8]

The 12 model inputs

For each signal, the backend calculates the same four statistics using observations at or before the scoring time. It does not use future interpolation or patient identifiers as inputs. [3]

Signal	Four derived features
Temperature	Latest value, mean, trend per minute, standard deviation
Oxygen saturation	Latest value, mean, trend per minute, standard deviation
Heart rate	Latest value, mean, trend per minute, standard deviation

Mean describes the recent level. **Trend** is the fitted rate of change. **Standard deviation** measures variation. Together, these distinguish signals that have similar current values but different recent behavior. [3]

03 / NEOGUARD AI

How a risk score is calculated

The saved model is a **logistic regression**, exported as JSON. The backend loads its feature names, training means, scales, learned weights, intercept and decision threshold. It does not retrain itself when a new reading arrives. Training is a separate offline process. [4,5]

Step	Operation
1. Normalize	Subtract each feature's saved training mean and divide by its saved scale. This puts different units on comparable scales.
2. Combine	Multiply the normalized features by their learned weights, add them, and add the model intercept.
3. Convert	Apply a sigmoid function to produce a score between 0 and 1.
4. Classify	A score of 0.75 or higher produces a prototype warning/high category. A lower score produces a low category. Neither category is a clinical verdict.

```
z = intercept + sum(weight_i * normalized_feature_i)

score = 1 / (1 + exp(-z))
```

A score of 0.80 does not establish an 80% clinical risk. Clinical probability calibration has not been demonstrated. The backend also returns the model version, scoring window and feature contributions. Contributions describe influence on the model's weighted sum, not proof of a medical cause. [2,4,6]

What the web dashboard shows

State	Meaning
Prototype score	A matching score exists for the latest eligible reading; the panel shows Warning or No warning and explanatory factors.
Insufficient data	No eligible score exists for the latest reading, often because qualified history is incomplete.
Stale / unmonitored	The latest reading is too old, or there is no active monitoring session.
Disabled / unavailable	Scoring is turned off, or its model cannot be loaded.
Threshold mismatch	Active server thresholds do not match those used by the saved model; scoring is paused.

The model version is **synthetic-v1-20261009**. Version/hash checks prevent silently replacing an already registered model version. AI failures occur after sensor data is committed, so they do not undo acknowledged readings. Threshold alert handling remains separate. [2,4,7]

04 / NEOGUARD AI

How the model was trained and tested

The training script generates **480 simulated sessions**, sampled every five seconds. There are 60 sessions for each of eight scenarios: normal, cooling, desaturation, combined change, recovery, motion artifact, near-threshold behavior and dropout. These are generated scenarios, not recordings from 480 patients. [5,6]

Training	Validation	Held-out testing
288 sessions (60%)	96 sessions (20%)	96 sessions (20%)

Whole sessions are separated between the three groups. The scaler is fitted on training data only. Logistic-regression weights are fitted to those training features. The operating threshold is selected on validation sessions using a trade-off between event sensitivity and false-warning episodes; the held-out test sessions are not used to select it. [5,6]

What counts as a training event?

The label is positive when the simulator's underlying temperature falls below **36.5°C** or oxygen saturation falls below **90%**, either now or within the next 120 seconds. Labels use the simulator's underlying values rather than a deliberately corrupted sensor observation. Incomplete future-label windows are excluded from training. [5]

Saved held-out results: simulation only

Measure	AI model	Threshold baseline
Row precision	93.8%	99.4%
Row recall	81.8%	76.7%
Events detected by onset	44/49 (89.8%)	25/49 (51.0%)
False warning episodes/hour	1.42	0.75
Median lead time, detected events	107.5 seconds	0.0 seconds

The test report contains 19,272 eligible rows, 49 event onsets and about 26.77 hours of eligible simulated monitoring. Precision is the fraction of positive row predictions that match the future-event label; recall is the fraction of positive rows detected. Event detection and warning-episode measures use different units from row metrics. [6]

HOW TO READ THE COMPARISON

The model detected more simulated events by onset and produced earlier warnings, but also generated more false-warning episodes than the threshold baseline. Lead time is measured only among detected events. Both results come from the same synthetic generator and do not establish performance on real people.

05 / NEOGUARD AI

What the prototype can and cannot claim

AI, display averaging and a green LED have different meanings. The current bench firmware can show green with **IN RANGE / ESTIMATE** when fresh displayed values are within its limits, even while the full AI-quality gate remains false. A 24 October bench test confirmed that behavior and real backend delivery; it did not demonstrate a clinically validated prediction. [8]

Bench device behavior	AI model contract
Local surface low alarm: below 32°C. Green also requires averaged HR 100-160 bpm and oxygen at least 90%, with no confirmed alarm/fault.	The saved simulated event label uses temperature below 36.5°C or oxygen below 90%. Changing the local bench limit does not retrain or recalibrate this model.
MLX90614 reports surface temperature; the OLED average improves readability.	Surface readings are not established as equivalent to the temperature variable represented in the simulator. That measurement mismatch needs validation before interpreting a model output.

Current limits and next development work

Signal quality first. The captured bench estimates did not pass full qualification. Improve and independently verify sensor measurements before expecting a useful AI score. [8]

Real-data validation remains necessary. Training and testing share a synthetic generator, events are threshold-derived, and the archive source-label benchmark is separate from the deployed model. Suitable independently labelled data, a matching measurement protocol, and checks of calibration, missed events, warning frequency and lead time are needed to evaluate future versions. [1,5,6]

Keep the layers independent. Local alarms continue without internet. Backend threshold alerts remain available when the AI is disabled or unavailable. The experimental score supports research review; it does not provide a diagnosis, a medical all-clear or evidence of clinical approval. [2,7,8]

Implementation references

Sources are project files reviewed for this report. Paths are relative to the NeoGuard repository. Saved metrics are reported, not newly rerun training results.

[1] Project overview. ai/README.md

[2] Scoring orchestration, availability and model registry. backend/app/services/risk.py

[3] Quality windows and 12-feature extraction. backend/app/services/risk_features.py

[4] Prediction function and saved model. backend/app/services/risk_model.py; ai/artifacts/risk_model.json

[5] Session simulation, training and validation selection. ai/train.py

[6] Saved evaluation and group split. ai/reports/training_report.json; ai/reports/split_manifest.json

[7] Sync/inference separation and web presentation. backend/app/api/v1/endpoints/devices.py; frontend/src/components/risk/RiskScorePanel.jsx

[8] Device qualification, averaging and bench evidence. akena firmware/akena_firmware/: measurement.h, live_vitals.h, offline_sync.h, HARDWARE_TEST.md